SIMATS ENGINEERING

CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1709

Register Number	**192511053**
Name	**B Santhosh Kumar**

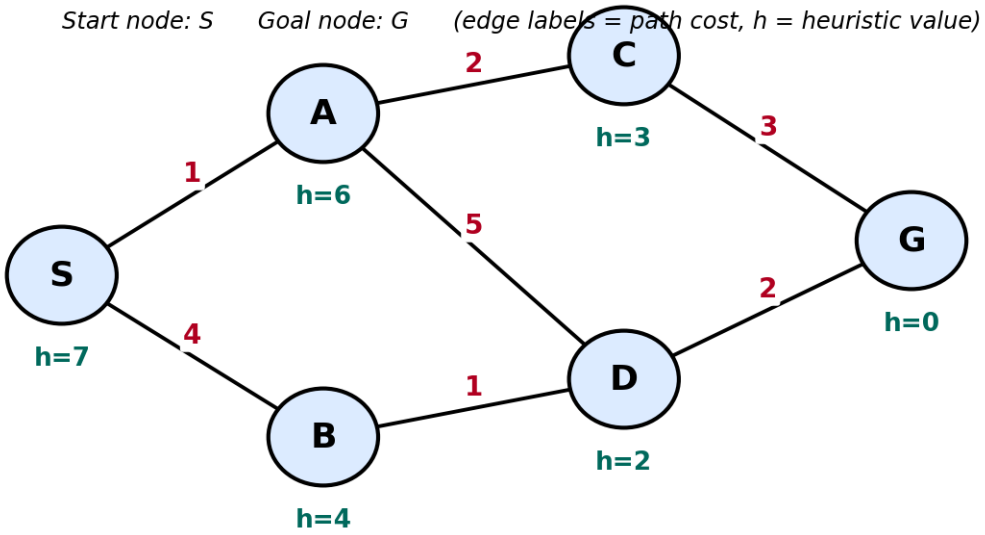
COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 20%

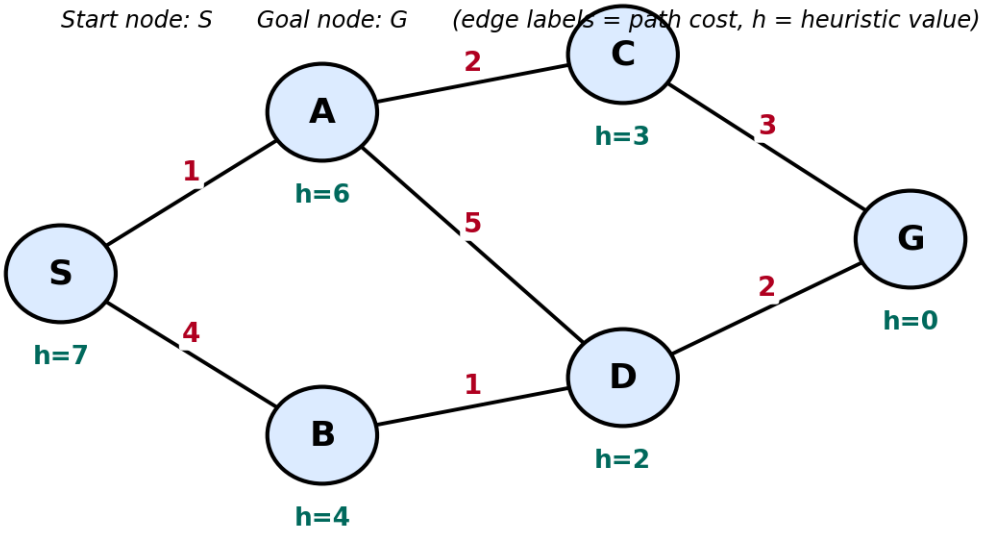
QUESTIONS: 5 **Total Marks:50**

Q1. Consider a suitable state-space graph (with heuristic values h(n) assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.

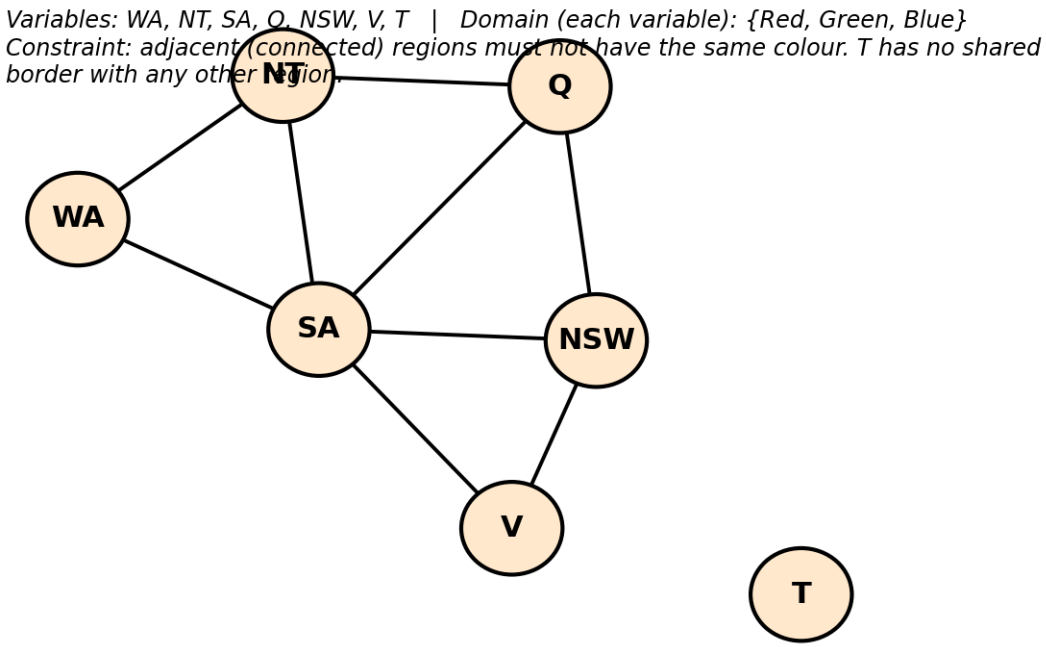


Q2. For a suitable weighted state-space graph (with edge costs and heuristic values h(n)) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the g(n), h(n) and f(n) values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.

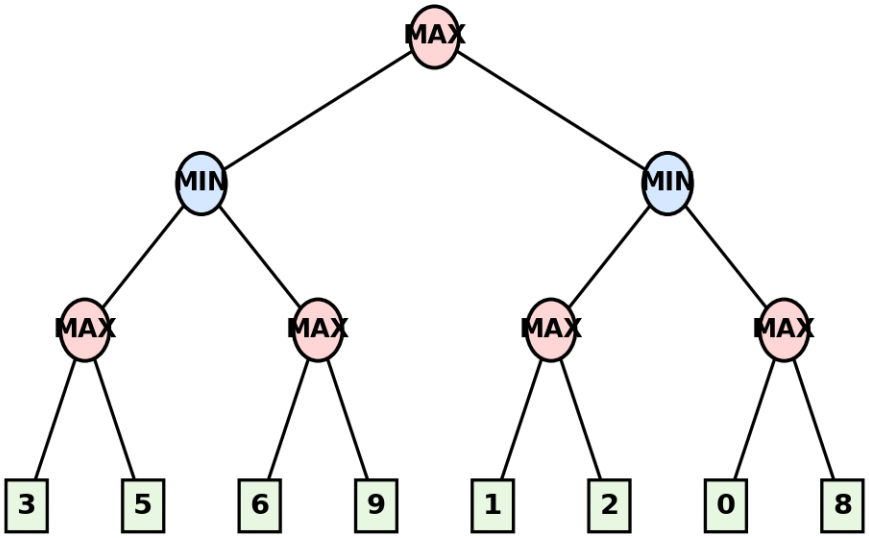


Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.



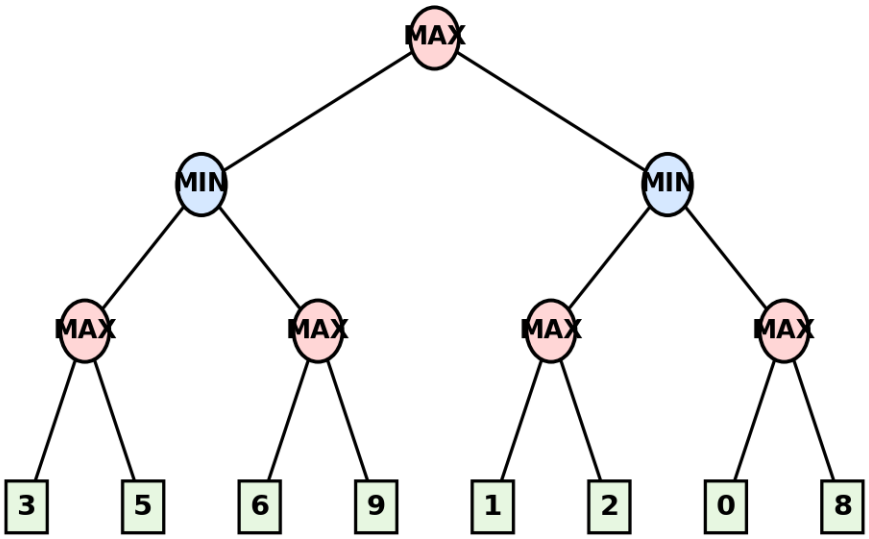
Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

*I **B Santhosh Kumar/192511053** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

B Santhosh Kumar
Signature of the student:

Dry Run Evaluation

RUBRICS FOR CALCULATION

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	**10**
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	**10**
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α–β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the	Multiple calculation errors are present that affect the correctness of	Calculations are mostly incorrect or missing.	**10**

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
		final outcome.	intermediate steps.		
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	**10**
Clarity of Presentation & Justification (trace tables/trees/ diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	**10**
				Total	50

Q1. Greedy Best-First Search (GBFS)

Introduction:

Greedy Best-First Search (GBFS) is an informed search technique used to find a path from the start node to the goal node. It selects the node with the smallest heuristic value **h(n)**, assuming it is closest to the goal. This method is simple and usually reaches the goal quickly, but it does not always produce the shortest path.

Problem Formulation:

Start Node: S

Goal Node: G

Heuristic Values

Node	h(n)
S	7
A	6
B	4
C	3
D	2
G	0

Edge Costs

Edge	Cost
S → A	1
S → B	4
A → C	2
A → D	5
B → D	1
C → G	3
D → G	2

State Space Graph

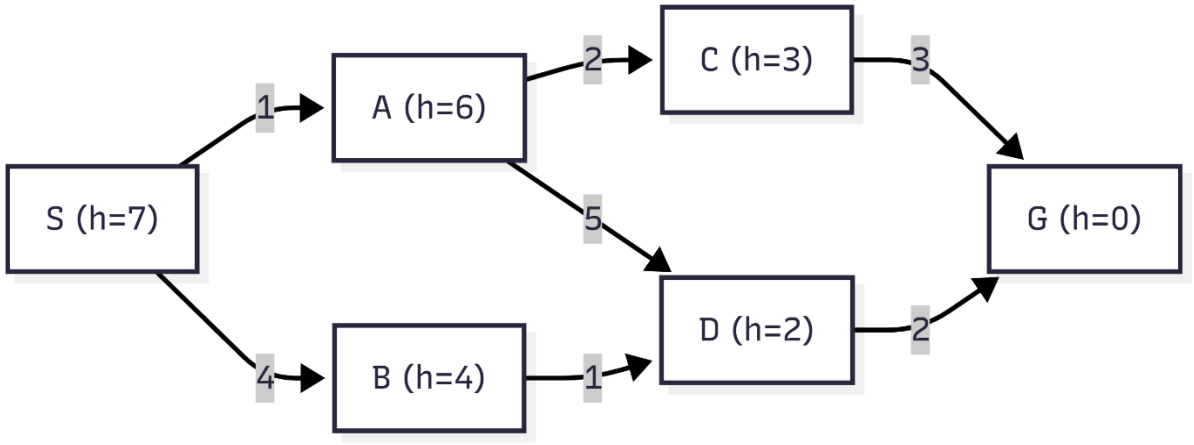


Figure 1: State Space Graph for Greedy Best-First Search

Source: Created by the author using Mermaid Diagram Generator, based on the assignment problem.

Dry Run:

Step 1

Current Node = **S**

Open List:

Node	**h(n)**
S	7

Expand **S**.

New nodes added:

- A (6)
- B (4)

Updated Open List:

Node	**h(n)**
B	4
A	6

Next selected node = **B**

Step 2

Current Node = **B**

Open List:

Node	h(n)
B	4
A	6

Expand **B**.

New node added:

- D (2)

Updated Open List:

Node	h(n)
D	2
A	6

Next selected node = **D**

Step 3

Current Node = **D**

Open List:

Node	h(n)
D	2
A	6

Expand **D**.

New node added:

- G (0)

Updated Open List:

Node	h(n)
G	0
A	6

Next selected node = **G**

Step 4

Current Node = **G**

Goal node reached. Search stops.

Node Expansion Order:

Step	Expanded Node
1	S
2	B
3	D
4	G

Final Path Obtained:

S → B → D → G

Path Cost

Path	Cost
S → B	4
B → D	1
D → G	2
Total Cost	**7**

Result:

The Greedy Best-First Search algorithm selects the node with the smallest heuristic value at each step. For the given graph, the nodes are expanded in the order **S → B → D → G**, and the goal node is reached successfully. The final path obtained is **S → B → D → G** with a total path cost of **7**

Q2. A* Search

Introduction:

A* Search is an informed search algorithm used to find the shortest path between a start node and a goal node. It selects the node with the smallest **f(n) = g(n) + h(n)** value. Here, **g(n)** is the path cost from the start node and **h(n)** is the heuristic value.

Problem Formulation:

Start Node: S

Goal Node: G

Heuristic Values

Node	h(n)
S	7
A	6
B	4
C	3
D	2
G	0

Edge Costs

Edge	Cost
S → A	1
S → B	4
A → C	2
A → D	5
B → D	1
C → G	3
D → G	2

State Space Graph:

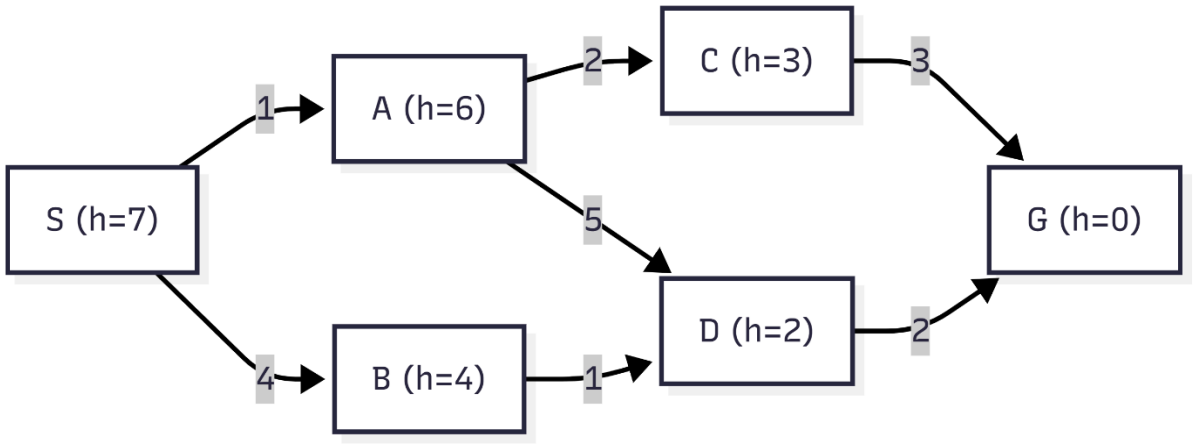


Figure 1: State Space Graph for A* Search

Source: Self-created using Mermaid based on the given assignment graph.

Dry Run

Step 1

Expand **S**

Node	g(n)	h(n)	f(n)=g+h
A	1	6	7
B	4	4	8

Choose **A** (lowest f = 7).

Step 2

Expand **A**

Node	g(n)	h(n)	f(n)
C	3	3	6
D	6	2	8
B	4	4	8

Choose **C** (lowest f = 6).

Step 3

Expand **C**

Node	g(n)	h(n)	f(n)
G	6	0	6
D	6	2	8
B	4	4	8

Choose **G** (lowest f = 6).

Goal node reached.

Node Expansion Order:

Step	Expanded Node
1	S
2	A
3	C
4	G

Final Optimal Path:

S → A → C → G

Total Path Cost:

Edge	Cost
S → A	1
A → C	2
C → G	3

Total Cost = 1 + 2 + 3 = 6

Result:

Using the A* Search algorithm, the goal node **G** is reached successfully. The nodes are expanded in the order **S → A → C → G**. The optimal path obtained is **S → A → C → G** with a total path cost of **6**.

Q3. Constraint Satisfaction Problem (Map Colouring using Backtracking Search)

Introduction:

A Constraint Satisfaction Problem (CSP) is a problem where values are assigned to variables while satisfying a given set of constraints. In the Australia Map Colouring problem, each state must be assigned a colour such that no two neighbouring states have the same colour. Backtracking Search is used to find a valid solution by trying different colour assignments whenever a conflict occurs.

Problem Formulation:

Variables:

WA, NT, SA, Q, NSW, V, T

Domain:

Each variable can take one of the following colours:

{Red, Green, Blue}

Constraints:

- Adjacent (connected) states cannot have the same colour.
- Tasmania (T) has no neighbouring state, so it can be assigned any colour.

Neighbouring States:

- WA → NT, SA
- NT → WA, SA, Q
- SA → WA, NT, Q, NSW, V
- Q → NT, SA, NSW
- NSW → SA, Q, V
- V → SA, NSW
- T → No neighbouring state

Constraint Graph:

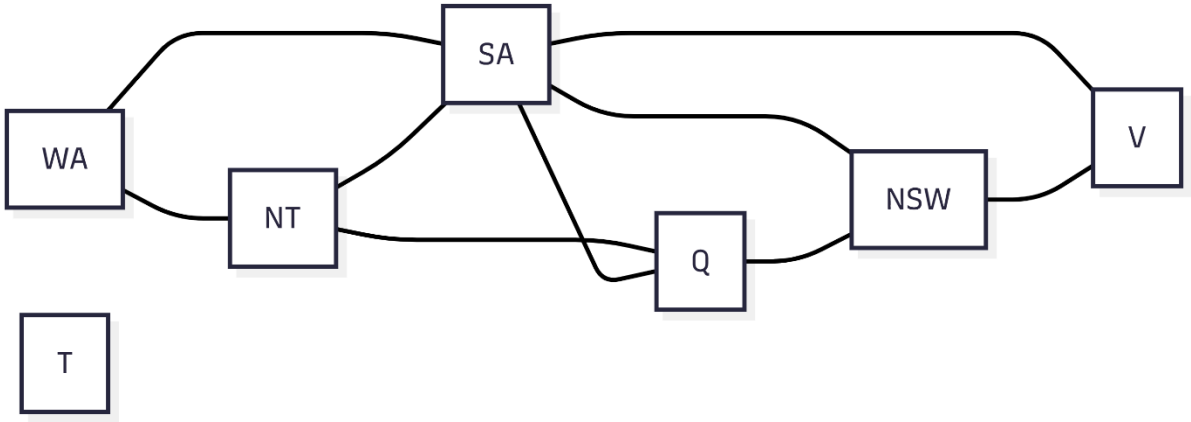


Figure 1: Constraint Graph for Australia Map Colouring

Source: Self-created using Mermaid based on the given assignment graph.

Dry Run of Backtracking Search:

Step 1

Assign **WA = Red**.

There are no previously assigned neighbouring states.

Assignment accepted.

Step 2

Assign **NT = Green**.

Check:

- WA = Red
- NT = Green

The neighbouring states have different colours.

Assignment accepted.

Step 3

Assign **SA = Red**.

Check:

- WA = Red
- SA = Red

Both neighbouring states have the same colour. This violates the constraint.

Backtrack and assign another colour.

Assign **SA = Blue**.

Now,

- WA = Red
- NT = Green
- SA = Blue

All neighbouring states have different colours.

Assignment accepted.

Step 4

Assign **Q = Green**.

Check:

- NT = Green
- Q = Green

The neighbouring states have the same colour.

Backtrack and assign another colour.

Assign **Q = Red**.

Now,

- NT = Green
- SA = Blue
- Q = Red

The assignment satisfies all constraints.

Step 5

Assign **NSW = Blue**.

Check:

- SA = Blue
- NSW = Blue

Both neighbouring states have the same colour.

Backtrack and assign another colour.

Assign **NSW = Green**.

Now,

- SA = Blue
- Q = Red
- NSW = Green

The assignment satisfies all constraints.

Step 6

Assign **V = Blue**.

Check:

- SA = Blue
- V = Blue

The neighbouring states have the same colour.

Backtrack and assign another colour.

Assign **V = Red**.

Now,

- SA = Blue
- NSW = Green
- V = Red

The assignment satisfies all constraints

Step 7

Assign **T = Green**.

Tasmania has no neighbouring states.

Assignment accepted.

Constraint Checking Summary:

Variable	First Assignment	Status	Final Assignment
WA	Red	Accepted	Red
NT	Green	Accepted	Green
SA	Red	Conflict with WA	Blue
Q	Green	Conflict with NT	Red
NSW	Blue	Conflict with SA	Green
V	Blue	Conflict with SA	Red
T	Green	Accepted	Green

Final Colour Assignment:

State	Colour
WA	Red
NT	Green
SA	Blue
Q	Red
NSW	Green
V	Red
T	Green

Result

The Backtracking Search algorithm successfully assigns colours to all the states while satisfying the given constraints. Whenever a conflict occurs, the algorithm backtracks and tries another colour until a valid solution is found.

Q4. Minimax Algorithm

Introduction:

The Minimax algorithm is used in two-player games to choose the best move. The MAX player tries to maximize the score, while the MIN player tries to minimize it. The utility values are calculated from the leaf nodes and propagated to the root.

Problem Formulation:

- **Root Node:** MAX
- **Level 1:** MIN
- **Level 2:** MAX
- **Leaf Nodes (Utility Values):** 3, 5, 6, 9, 1, 2, 0, 8

Game Tree:

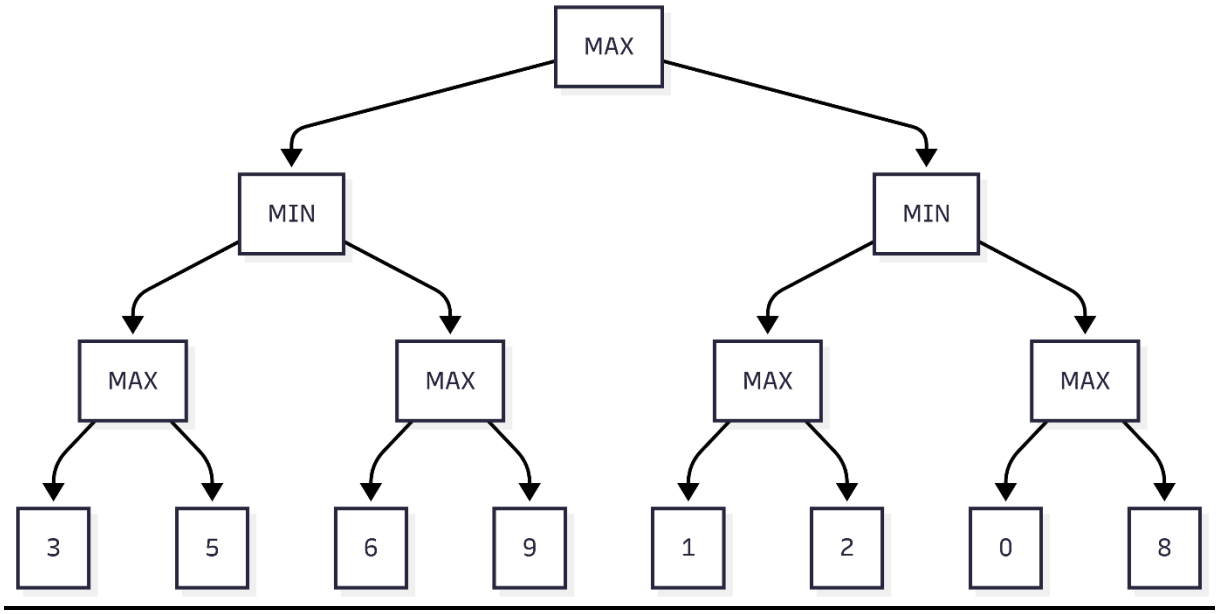


Figure 1: Game Tree for Minimax Algorithm

Source: Self-created using Mermaid based on the given assignment tree.

Dry Run of Minimax:

Step 1: Evaluate Leaf Nodes

Leaf node values are:

3, 5, 6, 9, 1, 2, 0, 8

Step 2: Evaluate MAX Nodes

The MAX player selects the larger value from each pair.

MAX Node	Leaf Values	Selected Value
MAX1	3, 5	**5**
MAX2	6, 9	**9**
MAX3	1, 2	**2**
MAX4	0, 8	**8**

Step 3: Evaluate MIN Nodes

The MIN player selects the smaller value from its child MAX nodes.

MIN Node	Child Values	Selected Value
MIN1	5, 9	**5**
MIN2	2, 8	**2**

Step 4: Evaluate Root Node (MAX)

The root MAX node compares:

- Left MIN = **5**
- Right MIN = **2**

MAX selects the larger value.

Root Value = 5

Backed-up Values:

Node	Value
MAX1	5
MAX2	9
MIN1	5
MAX3	2
MAX4	8
MIN2	2
Root (MAX)	**5**

Best Move:

The MAX player chooses the **left subtree**, because it gives the higher utility value.

Best Move: Root → Left MIN

Final Utility Value: 5

Result:

Using the Minimax algorithm, the utility values are propagated from the leaf nodes to the root. The final value at the root is **5**, so the MAX player selects the **left branch**, which gives the best possible outcome assuming the MIN player always plays optimally.

Q5. Minimax Algorithm with Alpha-Beta Pruning

Introduction:

Alpha-Beta Pruning is an optimization of the Minimax algorithm. It reduces the number of nodes evaluated by eliminating branches that cannot affect the final decision. This helps in finding the best move more efficiently.

Problem Formulation:

- **Root Node:** MAX
- **Level 1:** MIN
- **Level 2:** MAX
- **Leaf Nodes (Utility Values):** 3, 5, 6, 9, 1, 2, 0, 8

Game Tree:

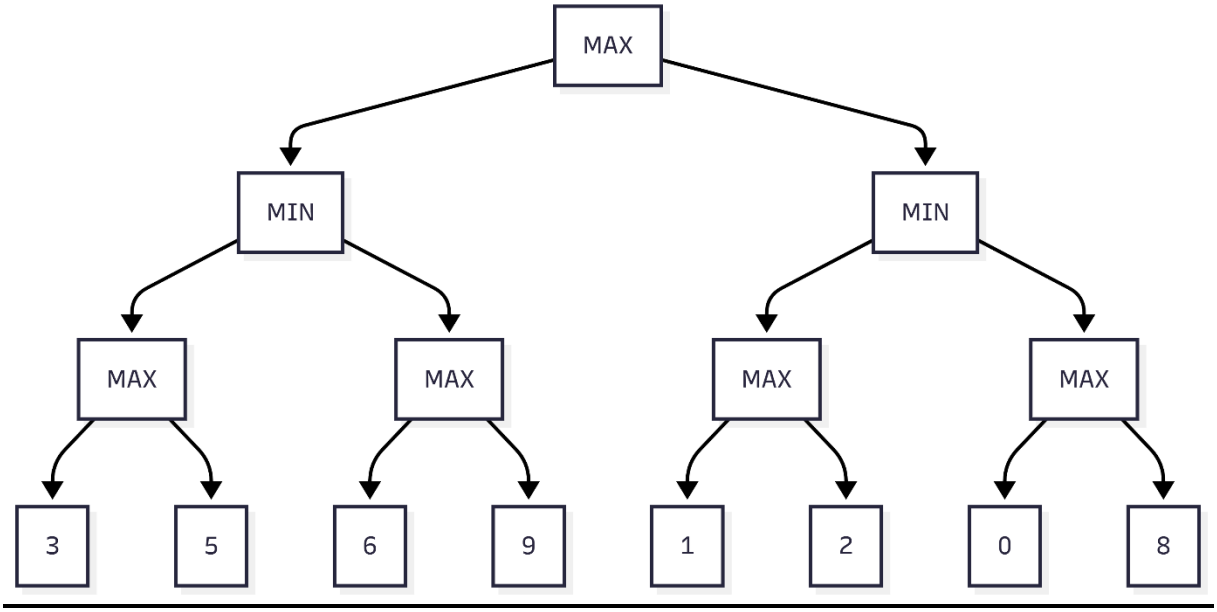


Figure 1: Game Tree for Alpha-Beta Pruning

Source: Self-created using Mermaid based on the given assignment tree.

Dry Run of Alpha-Beta Pruning:

Initially,

$\alpha = -\infty$, $\beta = +\infty$

Step 1

Evaluate the left MAX node.

Leaf values: **3, 5**

MAX selects **5**.

Return **5** to the left MIN node.

Now,

$\beta = 5$

Step 2

Evaluate the second MAX node.

Current values:

$\alpha = -\infty$, $\beta = 5$

First leaf = **6**

Now,

$\alpha = 6$

Since **α (6) $\geq$ β (5)**, the remaining leaf (**9**) is not evaluated.

This branch is **pruned**.

Left MIN node returns **5**.

Step 3

Root MAX receives value **5**.

Now,

$\alpha = 5$

Step 4

Move to the right MIN node.

Current values:

$\alpha = 5$, $\beta = +\infty$

Evaluate the third MAX node.

Leaf values: **1, 2**

MAX selects **2**.

Right MIN updates

$\beta = 2$

Since **β (2) $\leq$ α (5)**, the remaining right branch is not evaluated.

The subtree containing leaf values **0** and **8** is **pruned**.

Step 5

Root MAX compares

- Left subtree = **5**
- Right subtree = **2**

MAX selects **5**.

Alpha-Beta Values:

Node	**α**	**β**	**Value**
Root (MAX)	5	$+\infty$	5
Left MIN	$-\infty$	5	5
Right MIN	5	2	2

Pruned Branches:

Pruned Node	**Reason**
Leaf **9**	$\alpha = 6$ and $\beta = 5$, so $\alpha \geq \beta$
Subtree containing **0** and **8**	$\beta = 2$ and $\alpha = 5$, so $\beta \leq \alpha$

Final Best Move:

The MAX player chooses the **left subtree**.

Best Utility Value = 5

Result:

Using Alpha-Beta Pruning, the game tree is searched efficiently by pruning branches that cannot influence the final decision. The leaf node **9** and the subtree containing **0** and **8** are not evaluated. The best move for the MAX player is the **left subtree**, with a final utility value of **5**.